

PROJECT 2 — AIxAI NEGOTIATION SYSTEM

Interview Preparation — 50 Questions

A. Project Understanding (Q1–Q5)

Q1. Explain your AIxAI negotiation project. What does it do and what problem does it solve?

Answer:

- The project simulates a future scenario where AI agents negotiate SaaS subscription renewals automatically on behalf of customers and businesses.
 - The problem: In the current SaaS model, customer retention is reactive — companies offer discounts only when a customer is already leaving. There's no intelligent proactive negotiation.
 - Our system monitors customer behavior, predicts churn risk, and automatically initiates a negotiation between a Business AI agent and a Customer AI agent.
 - The Business AI tries to retain the customer with personalized offers. The Customer AI evaluates these offers based on the customer's actual preferences and utility score.
 - The system goes through multiple rounds of negotiation until either an agreement is reached or the customer decides to switch.
-

Q2. What is a "utility-based decision optimization system"? Why did you call it that?

Answer:

- "Utility-based" comes from decision theory — an agent makes decisions by calculating a utility score for each option and choosing the highest utility option.
- In our system, the Customer AI calculates a utility score for the current provider's offer based on weighted factors: price, features, reliability, security, loyalty, and switching cost.
- It also calculates utility scores for competing offers.

- The decision to renew or switch is based purely on whichever option gives the highest utility score — no LLM, no natural language, just mathematical optimization.
 - "Decision optimization" means the system is trying to optimize for the best decision given the customer's preferences and constraints.
-

Q3. What was your exact contribution? Which parts did YOU implement?

Answer:

- I designed and implemented the overall system architecture — the phase-by-phase workflow.
 - I implemented the Customer AI's utility calculation model — the weighted scoring function.
 - I implemented the Business AI's offer generation logic — how it generates and improves offers across rounds.
 - I built the churn prediction model — using usage data to estimate churn probability.
 - I implemented the negotiation loop — the multi-round back-and-forth between the two agents.
 - I implemented the reinforcement/profile update phase — updating customer preferences after a decision.
 - I built the REST API using FastAPI to expose the system's functionality.
-

Q4. Walk me through a complete negotiation from start to finish.

Answer:

1. **Monitor:** Business AI continuously tracks the customer's product usage, feature adoption, and renewal date.
2. **Risk Detection:** As the renewal date approaches, the churn prediction model calculates churn probability. If it crosses a threshold (e.g., 70%), negotiation is triggered.
3. **Customer Profile:** Customer AI has a preference model — weights assigned to price (30%), features (25%), reliability (15%), security (10%), loyalty (10%), switching cost (10%).
4. **Offer Generation:** Business AI generates an initial personalized offer — maybe a 10% discount and an additional feature.

5. **Utility Calculation:** Customer AI calculates the utility score of this offer vs. the best competitor's offer.
 6. **Decision:** If the utility of staying > utility of switching, Customer AI accepts. If not, it rejects.
 7. **Counter-offer:** Business AI improves the offer — maybe a 15% discount and two additional features. This continues for multiple rounds.
 8. **Final Decision:** After maximum rounds, Customer AI makes a final decision.
 9. **Reinforcement:** If the customer renews, their profile is updated — loyalty score increases, preferences are refined for future negotiations.
-

Q5. This project doesn't use LLMs or ChatGPT. Is it still "AI"?

Answer:

- Yes — AI is much broader than large language models.
 - This system uses AI concepts from classical AI and decision theory:
 - **Utility-based agents** — a formal AI agent architecture from Russell & Norvig's AIMA textbook.
 - **Churn prediction** — a machine learning model that estimates probability.
 - **Preference modeling** — learning and representing user preferences mathematically.
 - **Multi-round negotiation** — a game-theoretic approach to reaching agreement.
 - LLM-based agents would add natural language understanding — useful for chatbots but not necessary when the negotiation is structured and the preferences are numerical.
 - Rule-based + ML + utility functions is a legitimate, interpretable, and more controllable form of AI for this use case.
-

B. Architecture & System Design (Q6–Q12)

Q6. Describe the full architecture of your negotiation system.

Answer:

```
[Scheduler]
  ↓ (triggers before renewal date)
[Business AI]
  ■■■ Usage Monitor (tracks customer activity)
  ■■■ Churn Prediction Model (ML)
  ■■■ Offer Generator
  ■■■ Negotiation Engine
      ↑ REST API calls / function calls
[Customer AI]
  ■■■ Preference Model (weighted utility)
  ■■■ Competitor Comparison Module
  ■■■ Utility Calculator
  ■■■ Decision Maker

[Data Store]
  ■■■ Customer Profiles (usage, preferences, history)
  ■■■ Negotiation Logs
  ■■■ Subscription Records

[FastAPI Backend]
  ■■■ Exposes endpoints for triggering, monitoring, and results

[Reinforcement Module]
  ■■■ Updates customer profile post-decision
```

Q7. Why did you design this as two separate AI agents rather than one system making all decisions?

Answer:

- Separating concerns makes the system cleaner and more realistic — in a real future scenario, customer and business would each have their own autonomous AI.
- It enforces that the Business AI doesn't have direct access to the Customer AI's internal preferences — it can only observe behavior and outcomes, just like a real business can't read a customer's mind.
- It enables genuine multi-round negotiation — the Business AI must react to rejections without knowing exactly why they were rejected.

- Testing becomes cleaner — you can test the Customer AI independently to verify it makes rational decisions, and test the Business AI independently to verify it generates valid offers.
-

Q8. How does the churn prediction model work? What features does it use?

Answer:

- The churn prediction model is a binary classification model — output is a probability between 0 and 1.
- Features used:
 - Days since last active usage
 - Number of features actively used vs. total available
 - Number of support tickets filed recently
 - Days remaining until subscription renewal
 - Historical renewal behavior (did they renew last year?)
 - Competitor mentions or competitor page visits (if tracked)
- The model output is a churn probability (e.g., 0.72 = 72% likely to churn).
- If this exceeds a configurable threshold (e.g., 0.65), the negotiation workflow is triggered.

Interviewer Follow-up: What algorithm did you use for churn prediction?

Answer:

- This was not explicitly specified in the project description. In an interview, I would describe what I actually used. Common choices for this kind of tabular data classification would be logistic regression, decision trees, or random forest — all available in scikit-learn with NumPy/Pandas for data processing.
-

Q9. How does the offer generation work? Does the Business AI just randomly try different offers?

Answer:

- No — the offer generation is constraint-based and preference-aware.

- The Business AI has access to:
- The customer's historical usage data (what features they use most).
- The customer's tier and subscription value.
- Business constraints (minimum acceptable revenue, maximum discount allowed).
- Competitor benchmark prices.
- Round 1: Generate a moderate offer — a small discount + a feature they actually use.
- If rejected: Analyze the rejection (the Customer AI returns whether it was a price-based or feature-based rejection — or a general rejection if not enough information).
- Round 2: Improve the most impactful dimension — if the utility calculation suggests price sensitivity, increase the discount; if feature sensitivity, add more features.
- Maximum rounds: If no agreement after N rounds, the negotiation concludes.

Q10. What is the negotiation loop algorithm? How many rounds, and what determines when it stops?

Answer:

```

MAX_ROUNDS = configurable (e.g., 5)
round = 1
agreement = False

WHILE round <= MAX_ROUNDS AND NOT agreement:
    offer = BusinessAI.generate_offer(round, previous_rejections)
    customer_utility = CustomerAI.evaluate_offer(offer)
    competitor_utility = CustomerAI.evaluate_best_competitor()

    IF customer_utility >= competitor_utility:
        agreement = True
        CustomerAI.accept(offer)
    ELSE:
        BusinessAI.receive_rejection(offer, round)
        round += 1

IF NOT agreement:

```

```
CustomerAI.switch_to_competitor()
```

```
ReinforcementModule.update_profile(customer_id, agreement, final_offer)
```

- The loop stops when: Customer accepts an offer, or maximum rounds are exhausted.
- If exhausted without agreement, the customer switches.

Q11. How does the preference model work? Explain the utility calculation.

Answer:

- The utility function is a **weighted sum model**:

```
U(offer) =  $\sum$  (weight_i × score_i)
```

Where:

```
score_price = normalize(competitor_price / offer_price) -- higher sco
```

```
score_features = count(offer_features) / count(desired_features)
```

```
score_reliability = offer.uptime_sla / 100
```

```
score_security = security_rating / 5 -- on a 5-point scale
```

```
score_loyalty = customer_loyalty_score / 100
```

```
score_switching_cost = 1 - (switching_cost / max_switching_cost)
```

```
Weights: price=0.30, features=0.25, reliability=0.15,  
security=0.10, loyalty=0.10, switching_cost=0.10
```

- The Customer AI computes $U(\text{current_offer})$ and $U(\text{best_competitor_offer})$.
- If $U(\text{current_offer}) \geq U(\text{best_competitor_offer})$, the customer stays.
- Weights represent that customer's specific priorities — they can vary per customer profile.

Q12. Design a production version of this system for 1 million SaaS customers.

Answer:

Requirements: - Handle 1 million customers with varying renewal dates — thousands of negotiations per day. - Sub-second offer evaluation for good user experience. - Fault tolerance — a failed negotiation shouldn't corrupt the customer profile.

Architecture:

- **Scheduler:** A distributed task scheduler (Celery + Redis, or AWS Step Functions) triggers negotiation workflows as renewal dates approach.
- **Customer Service:** A microservice managing customer profiles, stored in PostgreSQL. Caches frequently accessed profiles in Redis.
- **Negotiation Engine:** A stateless microservice that takes customer profile and runs the negotiation loop. Can scale horizontally — multiple instances handle parallel negotiations.
- **Churn Prediction Service:** A separate ML inference service. Pre-computes churn scores nightly for all customers and stores them. Only re-computes if usage data changes significantly.
- **Message Queue:** Negotiations are queued via RabbitMQ/Kafka — prevents traffic spikes from overwhelming the negotiation engine.
- **Database:** PostgreSQL for customer profiles and negotiation history. Partitioned by customer_id for performance.
- **Caching:** Redis for competitor offer data (doesn't change minute-to-minute).
- **Monitoring:** Track negotiation success rates, average rounds to agreement, churn rates post-negotiation.

C. Technology Deep Dive (Q13–Q20)

Q13. Why did you use Python for this project? You have Java on your resume — why not Java?

Answer:

- Python was chosen because of its superior data science and ML ecosystem.
- NumPy and Pandas make numerical computation (utility functions, churn models, data manipulation) easy and efficient.

- scikit-learn provides ready-to-use ML algorithms for churn prediction without reinventing the wheel.
 - FastAPI is a Python framework with excellent async support and automatic API documentation.
 - Java would work, but its data science libraries are less mature and the development speed for a prototype/simulation system would be slower.
 - The choice was pragmatic — for data-heavy ML-adjacent work, Python is the right tool.
-

Q14. Why FastAPI over Flask or Django for this project?

Answer:

- **FastAPI vs Flask:**

- FastAPI has native async support (async/await) — important for handling concurrent negotiations.
- FastAPI auto-generates OpenAPI/Swagger documentation from type hints — very useful for a system with multiple API endpoints.
- FastAPI uses Python type hints for request/response validation — cleaner and more robust than Flask's manual validation.

Flask is more flexible but requires more boilerplate for what FastAPI provides out of the box.

- **FastAPI vs Django:**

- Django is a full-framework with ORM, admin panel, auth — overkill for an API-only simulation system.
- FastAPI is lightweight and focused — ideal when you just need fast API endpoints.

When would you still choose Flask?

- For very simple APIs where the overhead of FastAPI's type system isn't justified.
- When the team is already deeply familiar with Flask and the project is small.

When would you choose Django?

- When you need the full web framework — templates, admin panel, ORM, auth system all in one.
-

Q15. You used NumPy and Pandas. What specific tasks did each serve in this project?

Answer:

NumPy: - Used for the numerical operations in the utility calculation — array operations, weighted sums, normalization. - Used in the churn prediction model — feature arrays, matrix operations. - Efficient element-wise operations on customer feature vectors.

Pandas: - Used for loading and preprocessing customer usage data — reading CSV/tabular data, filtering, aggregating. - Computing feature statistics for the churn model — rolling averages, time-based aggregations. - Storing and manipulating negotiation history as DataFrames for analysis.

Interviewer Follow-up: Could you implement this without NumPy/Pandas?

Answer:

- Yes — pure Python lists and math operations could do everything NumPy/Pandas does for a small prototype.
- But NumPy is 10-100x faster for array operations because it uses vectorized C operations instead of Python loops.
- Pandas provides a much cleaner API for data manipulation than raw list comprehensions and dict operations.
- For a simulation processing many customer records, the performance and code clarity benefits are significant.

Q16. Explain the agent-based modeling approach. What is an "agent" in your system?

Answer:

- In agent-based modeling, an "agent" is an autonomous entity that:
- Has a set of possible actions.
- Has knowledge of its environment/state.
- Makes decisions based on that knowledge to achieve its goals.
- **Business AI agent:**
- Goal: Maximize customer retention within profit constraints.

- Actions: Generate offers, improve offers, give up after max rounds.
 - Environment knowledge: Customer usage data, churn score, business constraints, competitor prices.
 - **Customer AI agent:**
 - Goal: Maximize personal utility (get the best deal).
 - Actions: Evaluate offers, accept or reject, choose competitor.
 - Environment knowledge: Own preference weights, current offer, competitor offers.
 - These are **utility-based agents** — they act to maximize their utility function, not based on predefined rules or natural language.
-

Q17. You said "constraint-based optimization" — what constraints does the Business AI operate under?

Answer:

- **Revenue constraint:** The discount offered cannot make the deal unprofitable. There's a minimum revenue threshold per customer tier.
- **Feature constraint:** Some premium features cannot be given free to basic-tier customers — they must first be on the right plan.
- **Maximum discount constraint:** Business policy limits how much discount can be offered (e.g., max 30% discount).
- **Round constraint:** Maximum N rounds of negotiation. Beyond that, the cost of continued negotiation (agent compute time, engineering) exceeds the benefit.
- **Customer tier constraint:** Offers must be consistent with what that tier of customer is allowed to receive.

These constraints prevent the Business AI from making offers that are technically "accepted" by the customer but financially harmful to the business.

Q18. What REST API endpoints does your FastAPI backend expose?

Answer:

```
POST /negotiate/{customer_id}      -- Trigger negotiation for a customer
GET  /negotiate/{negotiation_id}   -- Get status/result of a negotiation
GET  /customers/{customer_id}      -- Get customer profile and preference m
PUT  /customers/{customer_id}      -- Update customer preferences/profile
GET  /customers/{customer_id}/churn_score -- Get current churn probability
GET  /negotiations                 -- List all negotiations (with filters)
GET  /health                       -- Health check endpoint
```

Interviewer Follow-up: How does FastAPI handle request validation?

Answer:

- FastAPI uses **Pydantic models** for request and response validation.
- You define a Python class with type-annotated fields. FastAPI automatically validates incoming request body against this schema.
- If validation fails, it returns a 422 Unprocessable Entity response with details about which fields failed and why.
- This catches bad input before it reaches your business logic.

Q19. How does the scheduler trigger negotiations? What technology did you use?

Answer:

- The scheduler monitors subscription renewal dates.
- N days before a renewal date (e.g., 30 days), if churn score is above threshold, it triggers negotiation.
- Implementation options:
 - **APScheduler (Python):** A job scheduler library that runs inside the FastAPI app. Simple, good for single-server setups.
 - **Celery + Redis:** A distributed task queue. Better for production — tasks are queued and multiple workers process them. Redis is the message broker.
 - **Cron job:** A system-level cron that hits the `/negotiate/{customer_id}` endpoint directly.

What was actually implemented: Not explicitly specified — in an interview, describe what you actually used.

Q20. Rule-based vs utility-based vs LLM-based negotiation — why utility-based?

Answer:

Approach	Advantage	Disadvantage
Rule-based	Simple, fast, fully predictable	Rigid, can't adapt to unknown scenarios
Utility-based	Mathematically optimal, interpretable, no LLM cost	Requires well-defined preferences, doesn't handle ambiguity
LLM-based	Flexible, handles natural language, can adapt	Expensive, slow, unpredictable, hard to audit

- For a structured SaaS negotiation where preferences are quantifiable (price, features, reliability), utility-based is the most appropriate.
- LLM-based would add cost and latency without improving the decision quality — the decision is mathematical, not linguistic.
- Rule-based would break down if customer preferences change or new offer types are introduced.
- Utility-based is the sweet spot: mathematically principled, interpretable, and efficient.

D. Code & Implementation (Q21–Q25)

Q21. Explain the core utility calculation function. Write it in Python.

Answer:

```

import numpy as np

class CustomerAgent:
    def __init__(self, preferences: dict):
        # preferences: {'price': 0.30, 'features': 0.25, ...}
        self.weights = preferences

    def calculate_utility(self, offer: dict) -> float:
        """
        offer: {
            'price_discount': 0.15,      # 15% discount
            'features': ['A', 'B', 'C'], # features included
            'reliability_sla': 99.9,      # uptime %
            'security_rating': 4,         # out of 5
        }
        Returns utility score between 0 and 1.
        """
        scores = {}

        # Price score: higher discount → higher score
        scores['price'] = offer.get('price_discount', 0)

        # Feature score: what fraction of desired features are offered
        desired = set(self.desired_features)
        offered = set(offer.get('features', []))
        scores['features'] = len(desired & offered) / len(desired) if desired

        # Reliability score: normalize SLA (99.9% → ~1.0)
        scores['reliability'] = offer.get('reliability_sla', 99.0) / 100.0

        # Security score: normalize rating
        scores['security'] = offer.get('security_rating', 3) / 5.0

        # Loyalty and switching cost (customer state, not offer dependent)
        scores['loyalty'] = self.loyalty_score / 100.0
        scores['switching_cost'] = 1.0 - (self.switching_cost / self.max_sw

```

```

        # Weighted sum
        utility = sum(self.weights[k] * scores[k] for k in self.weights)
        return utility

    def evaluate_offer(self, offer: dict) -> tuple[float, bool]:
        current_utility = self.calculate_utility(offer)
        competitor_utility = self.get_best_competitor_utility()
        accepted = current_utility >= competitor_utility
        return current_utility, accepted

```

Q22. How did you structure your Python modules? What does the file structure look like?

Answer:

```

negotiation_system/
├── main.py                # FastAPI app, route definitions
├── agents/
│   ├── business_agent.py  # BusinessAI class – offer generation
│   ├── customer_agent.py  # CustomerAgent class – utility calculation
│   └── negotiation_engine.py # Orchestrates the negotiation loop
├── models/
│   ├── churn_model.py     # Churn prediction logic
│   ├── preference_model.py # Customer preference data class
│   └── offer.py           # Offer data structure (Pydantic model)
├── data/
│   ├── customer_data.py   # Data loading and preprocessing
│   └── competitor_data.py  # Competitor offer data
├── scheduler/
│   └── renewal_scheduler.py # Scheduler logic
└── utils/
    └── normalizer.py       # Utility normalization functions

```

- OOP was applied throughout — each agent is a class with its own state and methods.
 - Separation of concerns — agents, models, data, and API are in separate modules.
-

Q23. How did you handle a situation where the negotiation reaches max rounds without agreement?

Answer:

```
def run_negotiation(customer_id: str) -> NegotiationResult:
    customer = load_customer_profile(customer_id)
    customer_agent = CustomerAgent(customer.preferences)
    business_agent = BusinessAgent(customer.usage_data, business_constraint)

    for round_num in range(1, MAX_ROUNDS + 1):
        offer = business_agent.generate_offer(round_num)
        utility, accepted = customer_agent.evaluate_offer(offer)

        if accepted:
            log_negotiation(customer_id, round_num, offer, "RENEWED")
            reinforce_profile(customer_id, offer, "RENEWED")
            return NegotiationResult(outcome="RENEWED", rounds=round_num, offer=offer)
        else:
            business_agent.record_rejection(offer, utility)

    # Max rounds exhausted
    log_negotiation(customer_id, MAX_ROUNDS, None, "CHURNED")
    reinforce_profile(customer_id, None, "CHURNED")
    return NegotiationResult(outcome="CHURNED", rounds=MAX_ROUNDS, offer=None)
```

- The result is always logged regardless of outcome.
- Profile reinforcement happens in both cases — if churned, the profile records it (which affects future interactions if the customer returns).

Q24. What data structures did you use and why?

Answer:

- **Dict (dictionary):** Used for customer preference weights (`{'price': 0.30, 'features': 0.25, ...}`) — fast key access, naturally maps feature names to weights.

- **NumPy arrays:** Used for batch computation of utility scores when evaluating multiple offers or running simulations across many customers.
- **Pandas DataFrame:** Used for customer usage data — tabular structure with time-indexed rows for usage metrics.
- **Pydantic models (dataclass-like):** Used for the `Offer` data structure — type-safe, validates data on creation, easily serialized to JSON for the API.
- **List/Queue:** Used for the negotiation history within a session — ordered sequence of offers and responses.

Q25. How did you debug the system when a negotiation was never reaching agreement?

Answer:

Systematic debugging:

Print the utility scores: Log `customer_utility` and `competitor_utility` for each round. If `customer_utility` is always lower, the offers aren't improving enough.

Check Business AI's offer improvement logic: Is the discount increasing each round? Is the logic correctly identifying what to improve?

Check the competitor baseline: If the competitor utility is set unrealistically high, no offer can beat it. Verify the competitor data is accurate.

Check preference weights: If price weight is 0.30 but the customer's switching cost is very high, they should still be willing to stay. Are the weights normalized correctly (sum to 1.0)?

Trace the acceptance condition: Is the condition `>=` or `>`? Off-by-one errors in comparisons can prevent acceptance at exact equality.

Unit test each component: Test `calculate_utility()` with known inputs and verify the output manually.

Add detailed logging: Log every offer, every utility score, every rejection reason to a file.

E. Database (Q26–Q30)

Q26. What is your database schema for this project?

Answer:

TABLE: customers

customer_id	VARCHAR PRIMARY KEY
name	VARCHAR
email	VARCHAR UNIQUE
subscription_tier	ENUM('BASIC', 'STANDARD', 'PREMIUM')
renewal_date	DATE
loyalty_score	DECIMAL(5,2) -- 0 to 100
switching_cost	DECIMAL(10,2) -- estimated cost to switch
created_at	TIMESTAMP

TABLE: customer_preferences

customer_id	VARCHAR PRIMARY KEY FK → customers
weight_price	DECIMAL(4,3) -- 0.30
weight_features	DECIMAL(4,3)
weight_reliability	DECIMAL(4,3)
weight_security	DECIMAL(4,3)
weight_loyalty	DECIMAL(4,3)
weight_switching_cost	DECIMAL(4,3)
updated_at	TIMESTAMP

TABLE: negotiations

negotiation_id	VARCHAR PRIMARY KEY
customer_id	VARCHAR FK → customers
started_at	TIMESTAMP
ended_at	TIMESTAMP
outcome	ENUM('RENEWED', 'CHURNED', 'IN_PROGRESS')
total_rounds	INT
final_offer_id	VARCHAR FK → offers

TABLE: offers

offer_id	VARCHAR PRIMARY KEY
negotiation_id	VARCHAR FK → negotiations
round_number	INT
discount_pct	DECIMAL(5,2)

```
features_offered JSON -- list of feature names
reliability_sla DECIMAL(6,3)
created_at      TIMESTAMP
```

```
TABLE: usage_metrics
```

```
metric_id      BIGINT AUTO_INCREMENT PRIMARY KEY
customer_id    VARCHAR FK → customers
recorded_at    TIMESTAMP
daily_active_minutes INT
features_used   JSON -- list of features used
support_tickets INT
```

Q27. Why SQL (relational) for this project? What about NoSQL?

Answer:

- **Why SQL:**

- The data is highly relational — customers link to preferences, negotiations link to customers and offers.
- Negotiation integrity matters — if an offer is written but the negotiation record fails, the data would be inconsistent. SQL transactions prevent this.

Queries like "which customers have churned in the last 30 days with loyalty score > 70?" are easy with SQL JOINS.

Where NoSQL makes sense:

- `usage_metrics` table could be better in a time-series store (InfluxDB) or even a document store (MongoDB) since usage data is append-heavy and the schema might vary.
- If the preference model becomes very complex (nested, varied structures), MongoDB documents could be more flexible.

Q28. How would you handle concurrent negotiations for the same customer?

Answer:

- A customer should not have two simultaneous active negotiations — this would create conflicting offers and profile updates.
 - **Database constraint:** Add a unique constraint on (`customer_id`, `outcome = 'IN_PROGRESS'`) in the negotiations table — only one in-progress negotiation per customer at a time.
 - **Application-level lock:** Before starting a negotiation, check if an active negotiation exists for that `customer_id`. If yes, return 409 Conflict.
 - **Optimistic locking:** Include a `version` field in the customer profile. If two processes try to update the same profile simultaneously, only the one with the correct version succeeds. The other gets a conflict error and retries.
 - **Distributed lock:** If running multiple negotiation service instances, use a Redis distributed lock with an expiry. Acquire the lock for `customer_id` before starting negotiation. Release it after.
-

Q29. How would you scale the database to handle 1 million customers and thousands of negotiations per day?

Answer:

- **Read replicas:** Add read replicas of the database. Dashboard queries, reporting, and customer profile reads go to replicas. Only writes go to the primary.
 - **Indexing:** Index `customer_id` on negotiations and `usage_metrics`. Index `renewal_date` on customers for efficient scheduler queries. Index `outcome` + `started_at` for reporting.
 - **Table partitioning:** Partition `usage_metrics` and `negotiations` by date (range partitioning). Old data in older partitions can be archived without affecting active data performance.
 - **Caching:** Cache customer profiles in Redis (TTL = 1 hour). Most negotiations read the same profile multiple times — caching avoids repeated DB reads.
 - **Archive old data:** Move completed negotiations older than 1 year to a data warehouse (like AWS Redshift) for analytics, keeping the operational DB lean.
-

Q30. Two processes try to update the same customer's preference weights simultaneously after a negotiation. What happens?

Answer:

- Without protection, this is a **lost update problem** — the last write wins and one update is silently discarded.
 - **Solution 1: Optimistic locking.** Add a `version` column to `customer_preferences`. Update query includes `WHERE version = :expected_version`. If 0 rows updated, the version changed — retry.
 - **Solution 2: Database transaction.** Wrap the read-then-update in a transaction with `SELECT FOR UPDATE` — this places a row-level lock on the customer's preference row for the duration of the transaction.
 - **Solution 3: Message queue serialization.** Post "update preferences" events to a queue. A single consumer processes them sequentially per `customer_id` — no concurrent writes at all.
 - In practice, post-negotiation updates are low frequency (one per negotiation, which can't overlap per customer), so a transaction with `SELECT FOR UPDATE` is sufficient.
-

F. API & Backend (Q31–Q34)

Q31. How does your FastAPI backend handle concurrent negotiation requests?

Answer:

- FastAPI is built on Starlette and uses Python's `async/await` with an event loop (uvicorn as the ASGI server).
- For I/O-bound operations (database queries, waiting for the next round's response), `async` allows handling many concurrent requests without blocking.
- The negotiation loop itself is CPU-bound (utility calculations) but lightweight enough that `async` handling works fine for moderate load.
- For production with high concurrency, run multiple uvicorn workers (e.g., `uvicorn main:app --workers 4`) and put a load balancer in front.

Interviewer Follow-up: Is the negotiation loop synchronous or async?

Answer:

- The negotiation loop is synchronous in terms of steps — round 1 must complete before round 2 starts. But the API endpoint handling it can be `async` — it awaits database

reads/writes between rounds.

Q32. How would you authenticate API calls to start a negotiation?

Answer:

- **JWT (JSON Web Tokens):** The dashboard or scheduler sends a JWT in the `Authorization: Bearer <token>` header.
 - The backend verifies the JWT signature and extracts the caller's identity and role.
 - Only an authenticated system (scheduler service) or admin user can call `POST /negotiate/{customer_id}`.
 - JWT has an expiry — short-lived tokens (1 hour) reduce the risk of stolen tokens.
 - For service-to-service calls (scheduler → negotiation service), use API keys or service account JWTs.
-

Q33. What happens if the churn prediction service is down when a negotiation is triggered?

Answer:

- The negotiation trigger depends on the churn score. If the churn service is unavailable:
 - **Option 1: Fail gracefully.** If churn score can't be obtained, skip the negotiation and retry later. Don't start a negotiation without the required data.
 - **Option 2: Use cached scores.** The scheduler pre-computes churn scores nightly. Use the cached score if the service is down. Accept slightly stale data.
 - **Option 3: Use a fallback threshold.** If churn score is unavailable but renewal is within 7 days, trigger negotiation anyway based on time proximity.
 - The system should log when fallback behavior is used so it can be monitored and the root cause addressed.
-

Q34. How do you handle idempotency — if the scheduler accidentally triggers the same negotiation twice for the same customer?

Answer:

- Before starting a negotiation, check the database for an existing `IN_PROGRESS` negotiation for that `customer_id`.
 - If one exists, return the existing `negotiation_id` instead of creating a duplicate.
 - Alternatively, use a unique constraint: `UNIQUE(customer_id, DATE(started_at))` — only one negotiation per customer per day.
 - The trigger call can be made idempotent by accepting a client-generated `request_id`. The backend stores processed `request_ids`. A duplicate request with the same `request_id` returns the original result without re-executing.
-

G. Security (Q35–Q38)

Q35. What are the security risks in this system?

Answer:

- **Data privacy:** Customer preference weights and usage data are sensitive business intelligence — must be encrypted at rest and in transit.
 - **Unauthorized negotiation trigger:** If the `POST /negotiate/{customer_id}` endpoint is not properly authenticated, anyone could trigger negotiations for any customer.
 - **Preference manipulation:** If an attacker can modify a customer's preference weights, they can make the customer accept any offer — or reject all offers to force churn.
 - **Offer manipulation:** If the Business AI's constraint parameters are tampered with, it could make offers that are financially harmful to the business.
 - **SQL injection:** If `customer_id` is used in raw SQL queries without parameterization, SQL injection is possible.
 - **API rate abuse:** An attacker could spam the negotiation endpoint to waste compute resources.
-

Q36. How would you protect customer preference data?

Answer:

- **Encryption at rest:** Database column encryption for sensitive fields (preference weights, usage data), or full-disk encryption at the database level.
 - **Encryption in transit:** All API calls use HTTPS (TLS). No plain HTTP.
 - **Access control:** Only the negotiation engine and authorized admin API can read preference data. Not exposed through public-facing endpoints.
 - **Audit logs:** Every read or write of preference data is logged with who did it and when. Anomalous access patterns trigger alerts.
 - **Data minimization:** Only store preference data that is necessary for the negotiation. Don't store raw customer behavioral data longer than needed.
 - **GDPR compliance:** Customers have the right to request deletion of their data — the system must be able to delete or anonymize customer profiles.
-

Q37. How do you prevent the Business AI from making financially harmful offers?

Answer:

- **Hard constraints in code:** The offer generator has minimum revenue thresholds that cannot be overridden. If an offer would result in below-minimum revenue, it's invalid and never generated.
 - **Input validation:** All business constraint parameters (max discount, minimum revenue) are server-side configuration, not client-provided. An API caller cannot change these constraints.
 - **Offer validation before sending:** Before the offer is sent to the Customer AI for evaluation, it passes through a validator that checks all business rules.
 - **Audit trail:** Every offer generated is logged. Financial impact of accepted offers is tracked and reported.
 - **Rate of change monitoring:** If the system's average discount starts drifting upward, an alert is sent for human review.
-

Q38. If a competitor's offer data in your system is tampered with (inflated artificially), what happens?

Answer:

- If competitor offers appear unrealistically good in the system, the Customer AI's `competitor_utility` score will be very high.
 - This means no Business AI offer can beat it — every negotiation results in CHURNED outcome.
 - This could be an attack to systematically trigger customer churn, or a data quality issue.
 - **Detection:** Monitor the negotiation success rate. A sudden drop (all negotiations ending in CHURNED) is a strong signal.
 - **Protection:** Competitor data should come from trusted, validated sources. Apply sanity checks — is this competitor price realistic? Is it within $\pm 30\%$ of historical prices?
 - **Alerting:** If competitor utility is above a threshold for all customers, raise an alert for manual review before continuing negotiations.
-

H. Failure & Edge Cases (Q39–Q42)

Q39. The negotiation never reaches agreement even for loyal customers who would normally renew. What would you debug?

Answer:

Systematic steps:

1. **Check competitor utility baseline:** Is the `competitor_utility` unrealistically high? Is the competitor data correct?
2. **Check the utility calculation:** Enable detailed logging for each round. Print `customer_utility` and `competitor_utility` side by side.
3. **Check preference weights:** Sum to 1.0? Verify with `sum(weights.values()) == 1.0`. Floating point errors can cause subtle issues.
4. **Check the acceptance condition:** Is there an off-by-one? `>=` vs `>`?
5. **Check the Business AI's improvement logic:** Is the discount actually increasing each round? Or is it stuck generating the same offer?
6. **Simulate manually:** Run a negotiation with hardcoded values and trace through the logic step by step.

7. **Check if loyalty score is being applied correctly:** Loyal customers have a loyalty boost — is this being added to their utility correctly?

Q40. What happens if the database is down when a negotiation tries to save its result?

Answer:

- The negotiation runs in memory and completes the logic.
 - At the save step, the database call fails.
 - The system should: 1. Retry the save with exponential backoff (3 retries: 1s, 2s, 4s). 2. If all retries fail, write the result to a fallback log file or an in-memory queue. 3. A background recovery job periodically drains the queue and saves to the database when it recovers. 4. Alert the operations team that database writes are failing.
 - The negotiation result must not be lost — the customer made a decision and the profile must eventually reflect it.
-

Q41. A customer's usage data is missing for the last 30 days. How does the churn model handle this?

Answer:

- Missing data is a known challenge for ML models. Strategies:
 - **Imputation:** Fill missing values with the customer's historical average or the global average for that tier.
 - **Missing indicator feature:** Add a binary feature "data_missing = True" so the model can learn from the absence pattern itself (missing data might itself be a churn signal — the customer stopped engaging).
 - **Default conservative behavior:** If data is insufficient for a reliable churn score, treat it as a moderate risk and trigger a review rather than automated negotiation.
 - **Do not use a stale prediction:** Trigger the churn model only if sufficient recent data exists. Log a warning if data is sparse.
-

Q42. What happens if the scheduler accidentally marks a wrong renewal date — triggering negotiation 90 days too early?

Answer:

- The negotiation runs, and the Business AI makes offers unnecessarily — potentially making commitments (discounts) that weren't needed since the customer wasn't planning to leave.
 - **Prevention:** Validate renewal dates against the source of truth (subscription billing system) before triggering negotiation. Never trust scheduler configuration alone.
 - **Detection:** An audit dashboard showing all triggered negotiations with the renewal date and days-until-renewal. Unusual patterns (many negotiations triggered > 60 days before renewal) would be visible.
 - **Graceful handling:** Add a check at negotiation start — if renewal is more than a configurable threshold (e.g., 45 days) away AND churn score is below moderate risk, abort and log a warning.
-

I. Improvement & Scalability (Q43–Q46)

Q43. What are the biggest limitations of your current simulation system?

Answer:

- **No real customer data:** The system uses simulated/synthetic customer data. A real deployment would need integration with the actual SaaS billing and usage systems.
 - **Static preference weights:** In the simulation, preference weights are predefined. In reality, they need to be inferred from customer behavior — a much harder problem.
 - **Simple churn model:** The current model is a prototype. A production churn model would need training on historical churn data, regular retraining, and careful evaluation.
 - **No A/B testing framework:** Can't currently test whether the negotiation system actually improves retention compared to no negotiation.
 - **Single-round competitor evaluation:** The Customer AI evaluates competitors based on static data. Real competitors change pricing and features dynamically.
-

Q44. How would you improve the preference model — making it learn from actual customer behavior?

Answer:

- **Behavioral inference:** Instead of hardcoding weights, infer them from behavior.
- A customer who always uses premium features → high feature weight.
- A customer who canceled after a price increase → high price weight.
- **Revealed preference theory:** Observe past decisions and use optimization to infer the preference weights that best explain historical choices.
- **Bayesian updating:** Start with prior weights and update them with each new piece of behavioral evidence.
- **Collaborative filtering:** Customers with similar usage patterns likely have similar preferences. Use clustering to group customers and infer weights from group behavior.

Note: These are **possible improvements**, not part of the original implementation.

Q45. How would you add LLM-based natural language negotiation on top of your utility-based system?

Answer:

- The utility-based system makes the *decision* (accept or reject, and why).
 - An LLM could handle the *communication* — generating a natural language offer message from the structured offer data.
 - Customer AI's rejection reason could be expressed in natural language rather than just a utility score.
 - Architecture: The utility engine remains the decision-maker. The LLM is a presentation layer that translates structured decisions into human (or agent-readable) language.
 - This keeps the decision logic interpretable and auditable while adding expressiveness to communication.
-

Q46. How would you add monitoring and analytics to measure whether this system actually improves retention?

Answer:

- **A/B test:** Randomly assign customers to "negotiation enabled" vs "control group" (no negotiation). Compare retention rates after 6 months.
 - **Negotiation funnel metrics:** Track at each phase — triggered → negotiation started → round 1 → round 2... → accepted/rejected. Where does the funnel drop off?
 - **Revenue impact:** Track revenue change — did customers who negotiated end up with higher or lower total spend?
 - **Model accuracy:** Track churn model accuracy — of the customers flagged as high churn, what percentage actually churned without negotiation?
 - **Dashboard:** A business intelligence dashboard showing negotiation success rates by customer tier, by offer type, by number of rounds.
-

J. Interviewer Pressure & Follow-ups (Q47–Q50)

Q47. You called this "AI" but it's just a weighted sum formula. Is this really AI?

Answer:

- This is a fair challenge. Let me address it directly.
 - A weighted sum utility function IS a form of AI — specifically, it implements a **utility-based rational agent**, which is a formal AI architecture from the academic AI literature.
 - The churn prediction model — if it uses logistic regression, decision trees, or any ML algorithm — is absolutely machine learning, which is a subset of AI.
 - "AI" does not require deep learning or LLMs. Classical AI includes search, optimization, planning, and utility maximization — all of which this system uses.
 - What I would NOT claim: this is a "neural network" or "deep learning" system or an "autonomous agent with cognition."
 - What I CAN claim: this is a rule-based + ML + utility optimization system that automates a decision-making process.
-

Q48. Couldn't you just build this as a simple if-then rule system? Why the complexity?

Answer:

- A rule system would look like: "If customer is on BASIC tier and hasn't logged in for 30 days, offer 10% discount."
- This works for simple cases but breaks down when:
- Customer preferences are different — one customer cares about price, another about features. A rule doesn't capture this.
- New customer types emerge — rules need to be manually updated.
- The number of possible situations grows — the rule list becomes unmanageable.
- The utility-based approach generalizes — it can handle any combination of preference weights and offer attributes without writing new rules.
- The churn model generalizes — it captures complex patterns in usage data that simple rules would miss.

Q49. You said the Business AI uses "reinforcement" to improve. Is this reinforcement learning?

Answer:

- No — and I need to be precise about this.
 - **Reinforcement Learning (RL)** is a specific ML paradigm where an agent learns a policy through trial and reward signals, typically using algorithms like Q-learning or policy gradient methods.
 - In my system, "reinforcement" refers to the simpler concept of **profile updating** — after a negotiation, the customer's profile (loyalty score, preference history) is updated to reflect the outcome.
 - This makes future negotiations more accurate — the system "remembers" what worked and what didn't for this customer.
 - It is NOT reinforcement learning in the RL sense — there is no reward function, no policy gradient, no value function.
 - I should always be precise about this in an interview — conflating profile updating with RL could mislead the interviewer.
-

Q50. If you had to completely redesign this project today, what would you change?

Answer:

- **Better preference inference:** Instead of predefined weights, build a behavioral model that infers preference weights from customer actions.
- **Real data integration:** Connect to an actual SaaS billing API and usage tracking system instead of simulated data.
- **Proper churn model evaluation:** Train on historical churn data, evaluate with precision/recall/AUC, and set up retraining pipelines.
- **A/B testing from day one:** Build the framework to compare negotiation vs. control groups.
- **Event-driven architecture:** Instead of a polling scheduler, use event-driven triggers — when usage drops or a competitor page is visited, trigger the churn assessment immediately.
- **Explainability:** Add a feature that tells the admin *why* a customer was flagged and *why* the offer was structured the way it was — full transparency.

Most Important 10 Questions (For Project 2)

1. Explain the project and the problem it solves.

(Answer: Q1 — future AI-to-AI SaaS negotiation, utility-based, churn prediction, multi-round negotiation)

2. Walk me through a complete negotiation flow.

(Answer: Q4 — Monitor → Churn → Profile → Offer → Utility → Decision → Reinforce)

3. Explain the utility calculation. How does the Customer AI decide?

(Answer: Q11/Q21 — weighted sum, normalize scores, compare to competitor utility)

4. Why FastAPI over Flask or Django?

(Answer: Q14 — async support, type validation, auto docs, lightweight)

5. Is this really "AI"? It's just a weighted sum.

(Answer: Q47 — utility-based agents are formal AI, churn model is ML, AI ≠ LLMs)

6. How did you handle concurrent negotiations for the same customer?

(Answer: Q28 — DB constraint, optimistic locking, Redis distributed lock)

7. What is the churn prediction model? What features does it use?

(Answer: Q8 — binary classifier, login frequency, feature adoption, ticket count, renewal proximity)

8. Is the "reinforcement" in your system actual Reinforcement Learning?

(Answer: Q49 — No. It's profile updating after negotiation, not RL with rewards and policy gradients)

9. How would you scale this to 1 million customers?

(Answer: Q12 — Celery scheduler, stateless negotiation engine, Redis cache, read replicas)

10. What would you change about this project today?

(Answer: Q50 — behavioral preference inference, real data, proper ML evaluation, A/B testing)

Questions I Must Never Get Wrong

The utility calculation formula — Know the weighted sum model. Know each factor and weight. Know how competitor comparison works. This is the core of the project.

Is this "AI"? — Utility-based agents are formal AI. Do NOT apologize for not using LLMs. Be confident and precise.

Is your "reinforcement" reinforcement learning? — Absolutely not. Profile updating ≠ RL. Conflating these is a red flag to a knowledgeable interviewer.

What does the churn prediction model do? — Binary classification, probability output, threshold-based trigger. Know the features and the algorithm (honestly).

What did FastAPI add that you couldn't get from Flask? — Async support, Pydantic validation, auto docs. Know these specifics.

What was YOUR contribution vs. what was library code? — Be specific about what you designed and coded, vs. what NumPy/Pandas/FastAPI did for you.

5-Minute Project Explanation

"The project is called AIxAI Negotiation System. The core idea is that in the future, AI agents might automatically negotiate subscription renewals on behalf of both the customer and the business — without any human in the loop.

The problem I was solving is that SaaS companies currently lose customers because they are reactive — they only offer discounts when a customer is already leaving. Our system is proactive. It monitors customer behavior, predicts churn risk using a machine learning model, and if the risk is high, automatically starts a negotiation.

The negotiation happens between two AI agents: the Business AI and the Customer AI. The Customer AI has a preference model — a weighted utility function where price is weighted 30%, features 25%, reliability 15%, and so on. The Business AI generates a personalized offer based on the customer's usage data and business constraints. The Customer AI evaluates the offer by calculating a utility score and comparing it to the best competitor's utility score. If our offer is better, the customer renews. If not, the Business AI improves the offer and tries again.

The system goes through multiple rounds — up to a configured maximum. If agreement is reached, the customer's profile is updated to make future negotiations more accurate.

I built this in Python using FastAPI for the API layer, NumPy for the utility calculations, Pandas for customer data processing, and a scheduler to trigger negotiations before renewal dates.

The biggest challenge was getting the offer improvement logic right — the Business AI had to intelligently decide which dimension to improve (price vs. features) based on limited feedback from the Customer AI.

If I were to improve it, I'd add behavioral preference inference so the weights are learned from customer actions rather than predefined, and I'd add a proper A/B testing framework to measure whether the system actually improves retention."